

***ECE391***  
***Computer System Engineering***  
*Lecture 10*



**Prof. Kirill Levchenko and Dong Kai Wang**

Slides Adapted from Prof. Kalbarczyk

University of Illinois at Urbana- Champaign

Spring 2024

# ECE391 EXAM 1



- **EXAM 1 – February 20 (Tuesday)**
  - Time: 7:00-9:00PM
  - Location: ECEB 1002 / 1015
  - *All content up to this lecture, all discussions so far, CN, PS1-2, MPs up to MP1 (no MP2 content).*
- **Conflict Exam**
  - Deadline to request conflict exam: **TODAY**
- **Exam 1 Review Session by TAs**
  - Date: Saturday February 17th
  - Time: 3:00-5:00PM
  - Location: ECEB 1002

# *Lecture Topics*

---

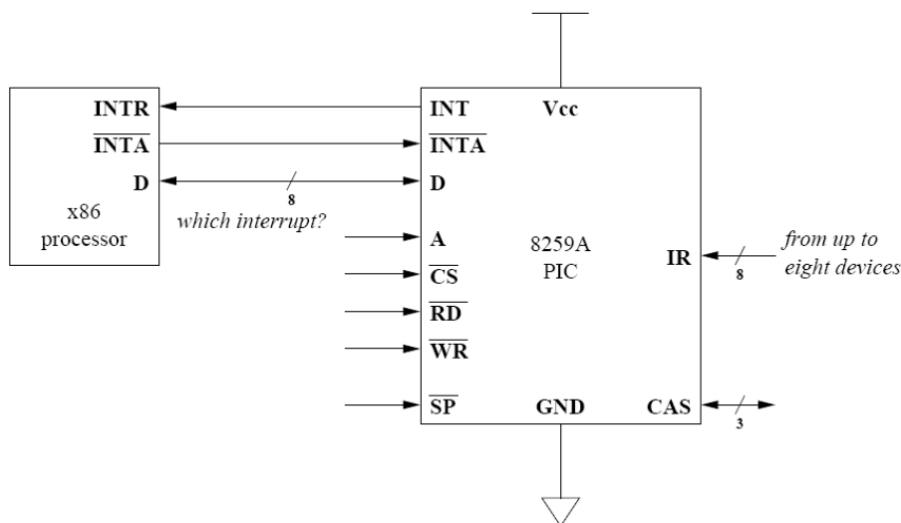
- Programmable Interrupt Controller (PIC)
  - Linux 8259A Initialization
  - Read the [8259A Datasheet](#)
- Linux Abstraction of PIC
- General Interrupt Abstractions
  - Interrupt Chaining
  - Soft Interrupts

# 8259A Operation in 8086 Mode

The events occur as follows in an MCS-80/85 system:

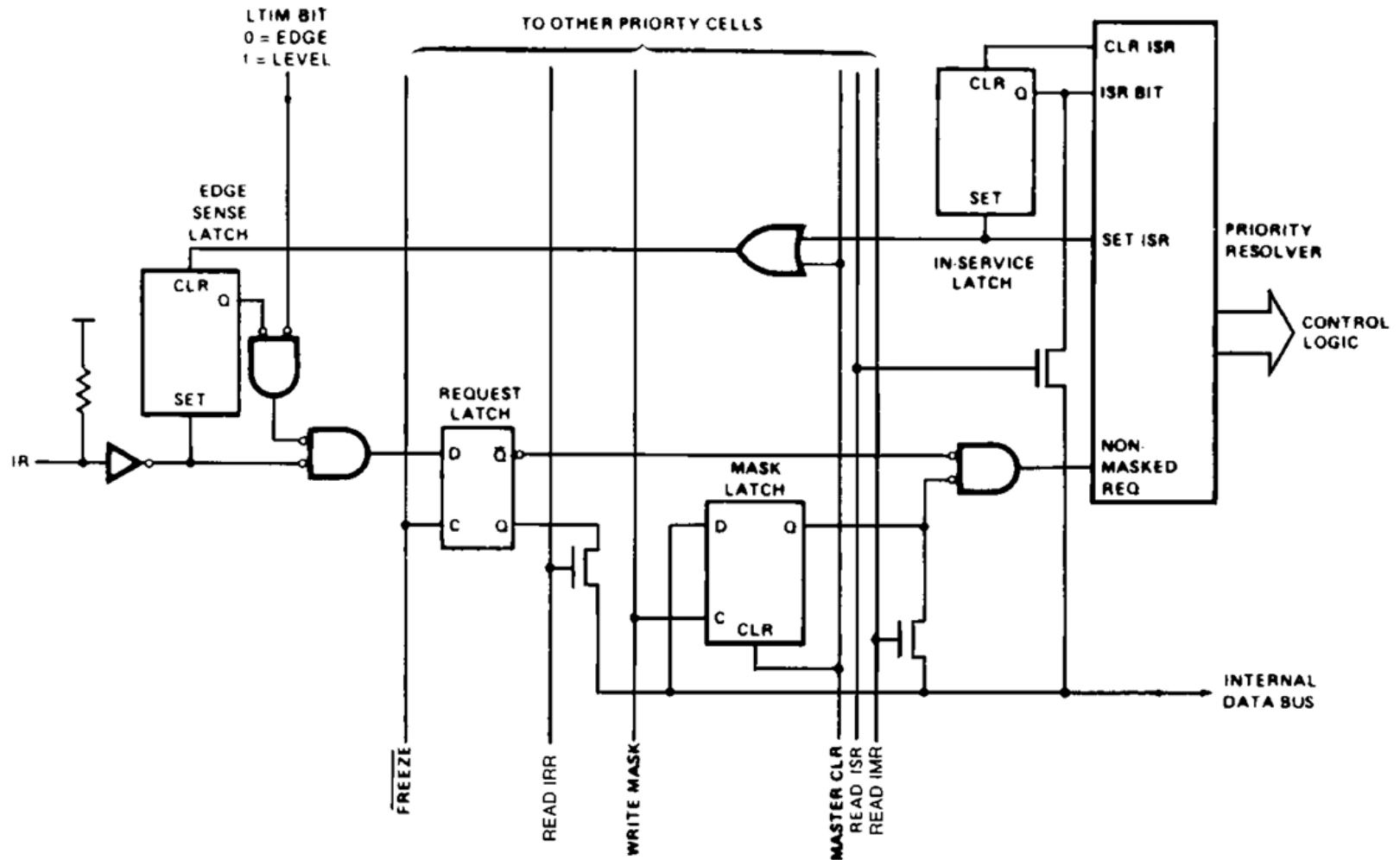
1. One or more of the INTERRUPT REQUEST lines (IR7–0) are raised high, setting the corresponding IRR bit(s).
2. The 8259A evaluates these requests, and sends an INT to the CPU, if appropriate.
3. The CPU acknowledges the INT and responds with an  $\overline{\text{INTA}}$  pulse.

4. Upon receiving an  $\overline{\text{INTA}}$  from the CPU group, the highest priority ISR bit is set and the corresponding IRR bit is reset. The 8259A does not drive the Data Bus during this cycle.
5. The 8086 will initiate a second  $\overline{\text{INTA}}$  pulse. During this pulse, the 8259A releases an 8-bit pointer onto the Data Bus where it is read by the CPU.
6. This completes the interrupt cycle. In the AEOL mode the ISR bit is reset at the end of the second  $\overline{\text{INTA}}$  pulse. Otherwise, the ISR bit remains set until an appropriate EOI command is issued at the end of the interrupt subroutine.





# 8259A Internal Hardware

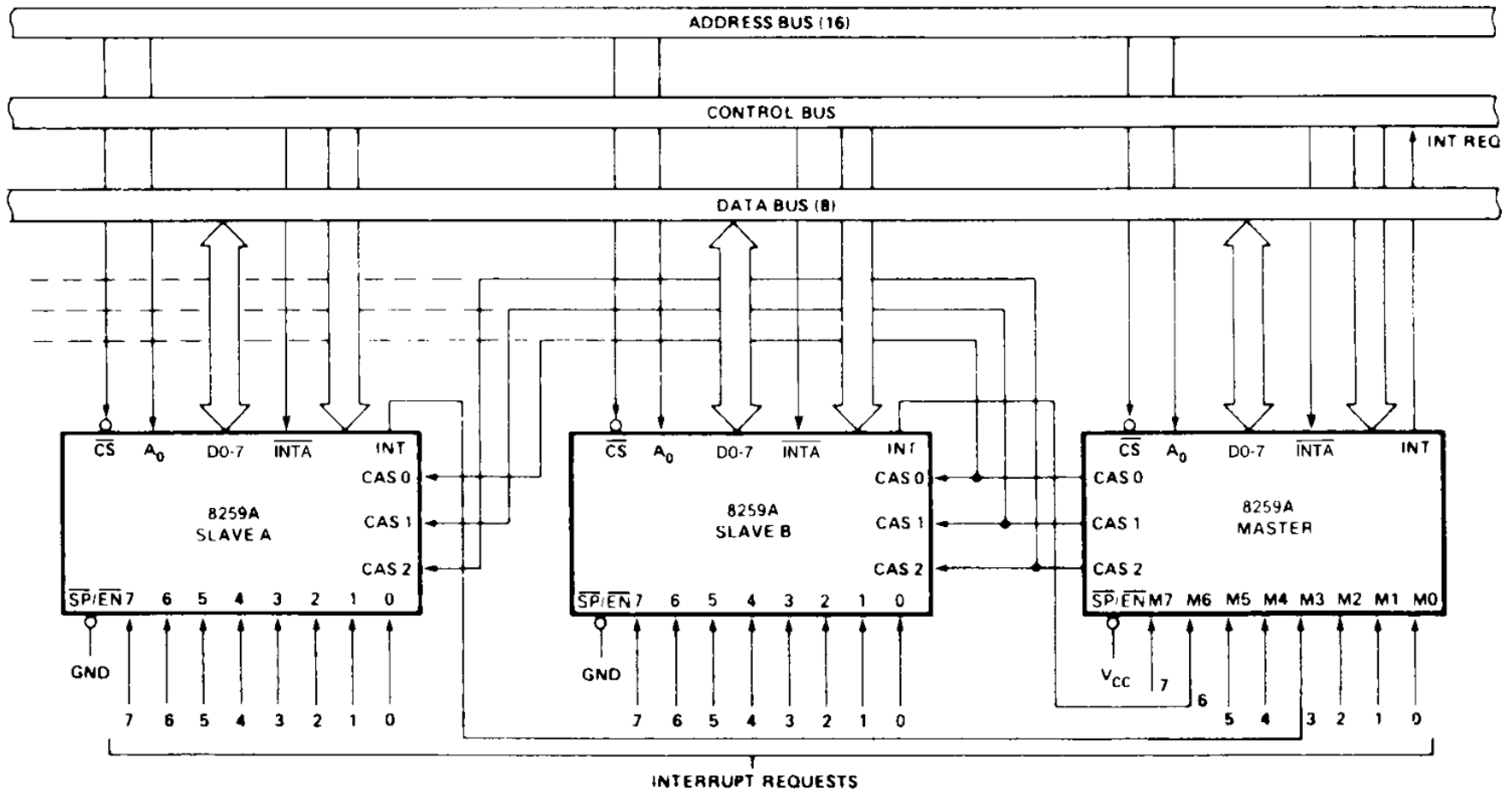


## *Logical Model of PIC Behavior (cont.)*

---

- What about A, CS', RD', and WR' ?
  - A = address match for ports.
  - CS' = chip select (does processor want PIC to read/write?).
  - RD' and WR' defined from processor's point of view:
    - RD' = processor will read data (vector #) from PIC.
    - WR' = processor will write data (command, EOI) to PIC.

# *Keep in Mind Address is Broadcasted*

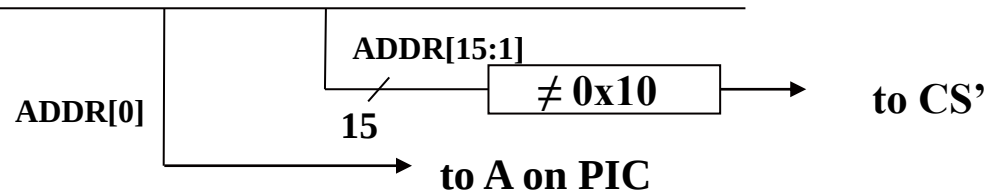




# Logical Model of PIC Behavior (cont.)

- Map to ports 0x20 & 0x21:
  - given ADDR bus.
  - logic to form A and CS' inputs to PIC.

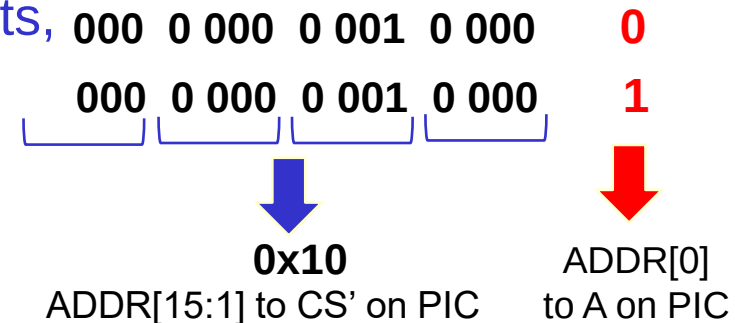
**ADDR bus**



- the PIC is asynchronous!
- all interactions have timing constraints,
- see specifications for details.

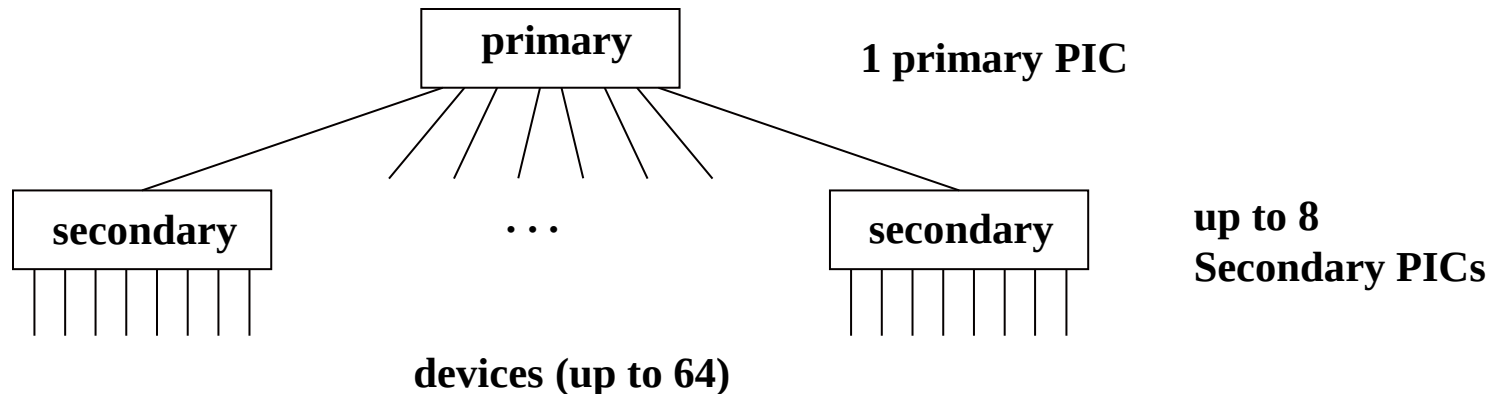
0x20 => 0000 0000 0010 000 **0**

0x21 => 0000 0000 0010 000 **1**

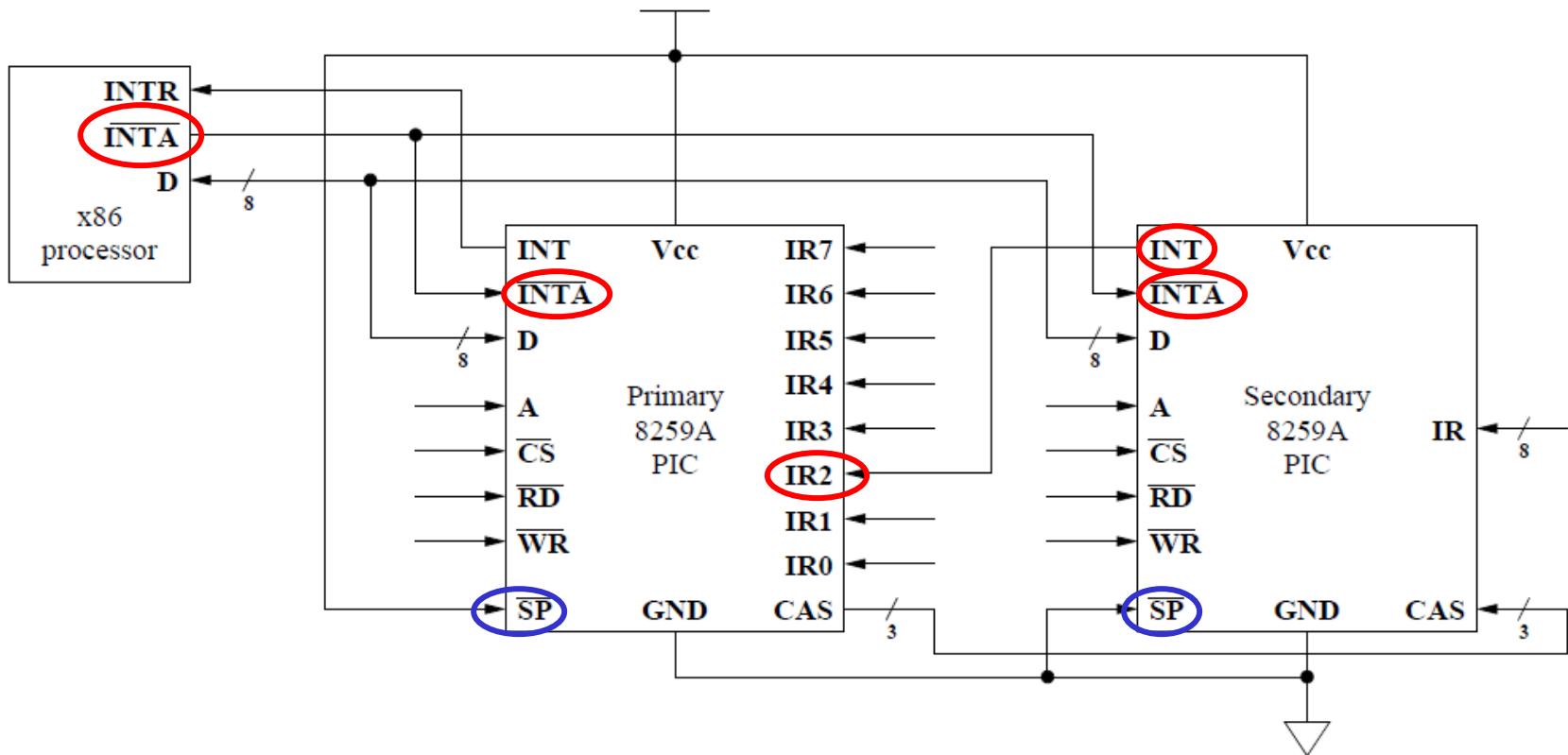


# *Logical Model of PIC Behavior (cont.)*

- What about SP' & CAS?
- Are eight devices enough? No? What then?
  - hook 2, 3, or 4 PICs to processor?  
[same problem as before!]
  - design a big PIC?  
[waste of transistors; not cheap in 8259A era]
  - design several PICs?  
[production costs]
- Better answer: **Cascade**



# Cascade Configuration of PICs



## *PIC (cont.)*

---

- Previous figure showed x86 configuration of two 8259A's
  - primary 8259A mapped to ports 0x20 & 0x21.
  - secondary 8259A mapped to ports 0xA0 & 0xA1.
  - secondary PIC connects to IR2 on primary PIC.
  - CAS acts as chip select for slaves (only master drives CAS).
  - processor must issue two EOIs. (*why?*)
- Question:
  - prioritization on 8259A: 0 is high, 7 is low.
  - what is prioritization across all 15 pins in x86 layout?
  - (highest) P0...P1...S0...S7...P3...P7 (lowest)

# PIC Interrupt Vectors

- PIC must write Interrupt Vector:
  - unique for each interrupt wire.
  - primary PIC IR's mapped to vector #'s 0x20 – 0x27.
  - secondary PIC IR's mapped to vector #'s 0x28 – 0x2F.
  - recall the IDT? (interrupt only tells you a device needs attention!)
- Vector from PIC:
  - 8-bit / 1-byte.
  - put on data line after 2nd INTA.
  - same upper 5 bits (T7-T3).
  - set on initialization.
- *What is T7-T3 here set to?*

Content of Interrupt Vector Byte  
for 8086 System Mode

|     | D7 | D6 | D5 | D4 | D3 | D2 | D1 | D0 |
|-----|----|----|----|----|----|----|----|----|
| IR7 | T7 | T6 | T5 | T4 | T3 | 1  | 1  | 1  |
| IR6 | T7 | T6 | T5 | T4 | T3 | 1  | 1  | 0  |
| IR5 | T7 | T6 | T5 | T4 | T3 | 1  | 0  | 1  |
| IR4 | T7 | T6 | T5 | T4 | T3 | 1  | 0  | 0  |
| IR3 | T7 | T6 | T5 | T4 | T3 | 0  | 1  | 1  |
| IR2 | T7 | T6 | T5 | T4 | T3 | 0  | 1  | 0  |
| IR1 | T7 | T6 | T5 | T4 | T3 | 0  | 0  | 1  |
| IR0 | T7 | T6 | T5 | T4 | T3 | 0  | 0  | 0  |

# Interrupt Descriptor Table (IDT)

|                                        |      |                                                      |                                       |
|----------------------------------------|------|------------------------------------------------------|---------------------------------------|
| 0x00–0x1F<br><br>defined<br>by Intel   | 0x00 | division error                                       |                                       |
|                                        | ⋮    |                                                      |                                       |
|                                        | 0x02 | NMI (non-maskable interrupt)                         |                                       |
|                                        | 0x03 | breakpoint (used by KGDB)                            |                                       |
|                                        | 0x04 | overflow                                             |                                       |
|                                        | ⋮    |                                                      |                                       |
|                                        | 0x0B | segment not present                                  |                                       |
|                                        | 0x0C | stack segment fault                                  |                                       |
|                                        | 0x0D | general protection fault                             |                                       |
|                                        | 0x0E | page fault                                           |                                       |
|                                        | ⋮    |                                                      |                                       |
| 0x20–0x27<br><br>primary<br>8259 PIC   | 0x20 | IRQ0 — timer chip                                    | example<br>of<br>possible<br>settings |
|                                        | 0x21 | IRQ1 — keyboard                                      |                                       |
|                                        | 0x22 | IRQ2 — (cascade to secondary)                        |                                       |
|                                        | 0x23 | IRQ3                                                 |                                       |
|                                        | 0x24 | IRQ4 — serial port (KGDB)                            |                                       |
|                                        | 0x25 | IRQ5                                                 |                                       |
|                                        | 0x26 | IRQ6                                                 |                                       |
|                                        | 0x27 | IRQ7                                                 |                                       |
| 0x28–0x2F<br><br>secondary<br>8259 PIC | 0x28 | IRQ8 — real time clock                               |                                       |
|                                        | 0x29 | IRQ9                                                 |                                       |
|                                        | 0x2A | IRQ10                                                |                                       |
|                                        | 0x2B | IRQ11 — eth0 (network)                               |                                       |
|                                        | 0x2C | IRQ12 — PS/2 mouse                                   |                                       |
|                                        | 0x2D | IRQ13                                                |                                       |
|                                        | 0x2E | IRQ14 — ide0 (hard drive)                            |                                       |
|                                        | 0x2F | IRQ15                                                |                                       |
| 0x30–0x7F                              | ⋮    | APIC vectors available to device drivers             |                                       |
| 0x80                                   | 0x80 | system call vector (INT 0x80)                        |                                       |
| 0x81–0xEE                              | ⋮    | more APIC vectors available to device drivers        |                                       |
| 0xEF                                   | 0xEF | local APIC timer                                     |                                       |
| 0xF0–0xFF                              | ⋮    | symmetric multiprocessor (SMP) communication vectors |                                       |

# Avoid Confusion!

## Port Address vs Interrupt Vector

- **Port:** used by in / out instructions to communicate with devices on I/O bus.
  - 0x20-21 for first PIC, 0xA0-A1 for second PIC.
- **Interrupt vector:** used as offset to IDT for specific interrupts. (0x20-27 for first PIC, 0x28-2F for second PIC)

| Port range    | Summary                                                                                                            |
|---------------|--------------------------------------------------------------------------------------------------------------------|
| 0x0000-0x001F | The first legacy <a href="#">DMA controller</a> , often used for transfers to floppies.                            |
| 0x0020-0x0021 | The first <a href="#">Programmable Interrupt Controller</a>                                                        |
| 0x0022-0x0023 | Access to the Model-Specific Registers of Cyrix processors.                                                        |
| 0x0040-0x0047 | The <a href="#">PIT</a> (Programmable Interval Timer)                                                              |
| 0x0060-0x0064 | The " <a href="#">8042</a> " <a href="#">PS/2 Controller</a> or its predecessors, dealing with keyboards and mice. |
| 0x0070-0x0071 | The <a href="#">CMOS</a> and <a href="#">RTC</a> registers                                                         |
| 0x0080-0x008F | The <a href="#">DMA</a> (Page registers)                                                                           |
| 0x0092        | The location of the fast <a href="#">A20</a> gate register                                                         |
| 0x00A0-0x00A1 | The second <a href="#">PIC</a>                                                                                     |

# Initialization Words

- 8259A accepts two types of command words:
  - Initialization **C**ommand **W**ords (**ICW**)
  - Operation **C**ommand **W**ords (**OCW**)
- Use the x86 in/out instructions to write these words.
  - Any command with A = 0 (writing to 0x20) and D4 = 1 is recognized as ICW1.
  - An initial write to A = 1 (writing to 0x21), after initialization, modifies masks registers.

## Operation Control Words (OCWs)

| OCW1 |    |    |    |    |    |    |    |    |
|------|----|----|----|----|----|----|----|----|
| A0   | D7 | D6 | D5 | D4 | D3 | D2 | D1 | D0 |
| 1    | M7 | M6 | M5 | M4 | M3 | M2 | M1 | M0 |

- Follow along the Linux src:  
<https://elixir.bootlin.com/linux/2.4.28/source/arch/i386/kernel/i8259.c>



# Linux 8259A Initialization

```
void init_8259A(int auto_eoi)
{
    unsigned long flags;

    i8259A_auto_eoi = auto_eoi;
    spin_lock_irqsave(&i8259A_lock, flags);

    outb(0xff, 0x21);      /* mask all of 8259A-1 */
    outb(0xff, 0xA1);      /* mask all of 8259A-2 */

    /*
     * outb_p - this has to work on a wide range of PC hardware.
     */
    outb_p(0x11, 0x20);    /* ICW1: select 8259A-1 init */
    outb_p(0x20 + 0, 0x21); /* ICW2: 8259A-1 IR0-7 mapped to 0x20-0x27 */
    outb_p(0x04, 0x21);    /* 8259A-1 (the primary) has a secondary on IRQ2
    if (auto_eoi)
        outb_p(0x03, 0x21);
    else
        outb_p(0x01, 0x21);

    outb_p(0x11, 0xA0);    /* ICW1: select 8259A-2 init */
    outb_p(0x20 + 8, 0xA1); /* ICW2: 8259A-2 IR0-7 mapped to 0x28-0x2f */
    outb_p(0x02, 0xA1);    /* 8259A-2 is a secondary on primary's IRQ2
    outb_p(0x01, 0xA1);

    if (auto_eoi)
        /*
         * in AEOI mode we just have to mask the interrupt
         * when acking.
         */
        i8259A_irq_type.ack = disable_8259A_irq;
    else
        i8259A_irq_type.ack = mask_and_ack_8259A;

    udelay(100);          /* wait for 8259A to initialize */

    outb(cached_21, 0x21);
    outb(cached_A1, 0xA1);

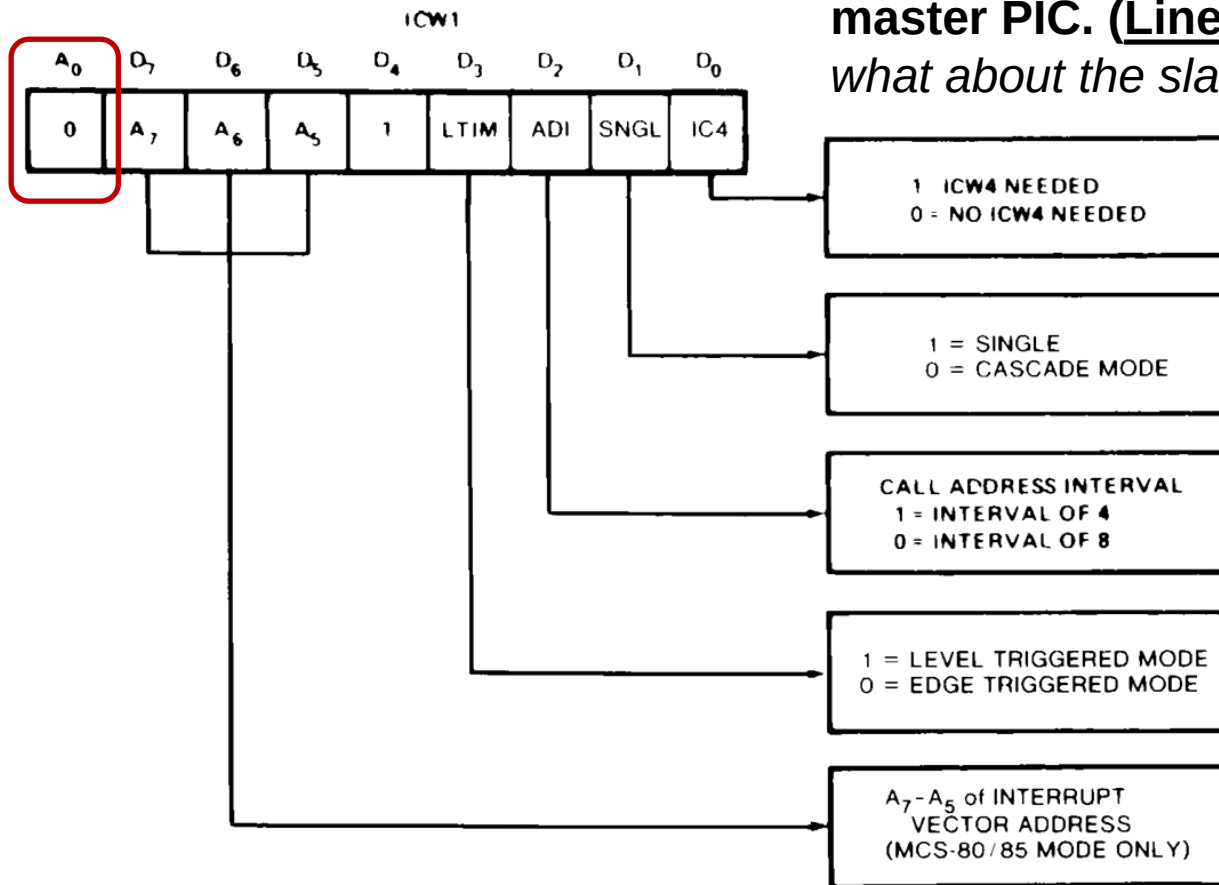
    spin_unlock_irqrestore(&i8259A_lock, flags);
}
```

# ICW1

```
outb_p(0x11, 0x20);    /* ICW1: select 8259A-1 init */
```

0 0 0 1 0 0 0 1

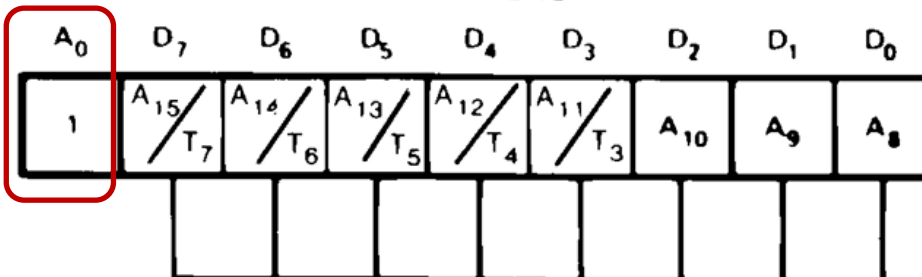
Write 0x11 to port 0x20 for master PIC. (Line 349)  
what about the slave PIC?



# ICW2

0 0 1 0 0 0 0 0

ICW2



Write 0x20 to 0x21 for  
master PIC. (Line 350)  
(*what about the slave PIC?*)

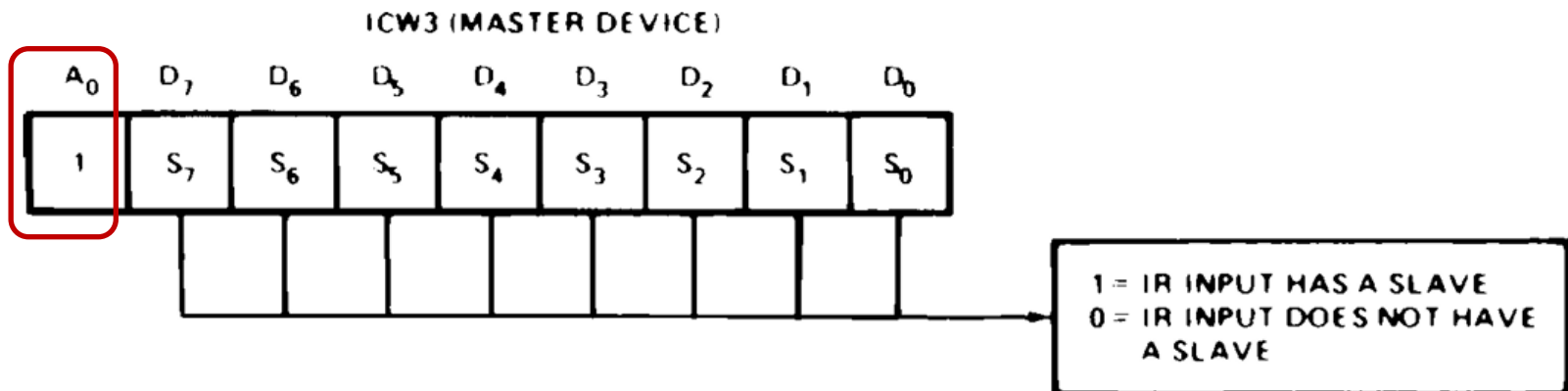
A<sub>15</sub>-A<sub>8</sub> OF INTERRUPT  
VECTOR ADDRESS  
(MCS80/85 MODE)  
T<sub>7</sub>-T<sub>3</sub> OF INTERRUPT  
VECTOR ADDRESS  
(8086/8088 MODE)

```
outb_p(0x20 + 0, 0x21); /* ICW2: 8259A-1 IR0-7 mapped to 0x20-0x27 */
```

# ICW3 Master

0 0 0 0 0 1 0 0

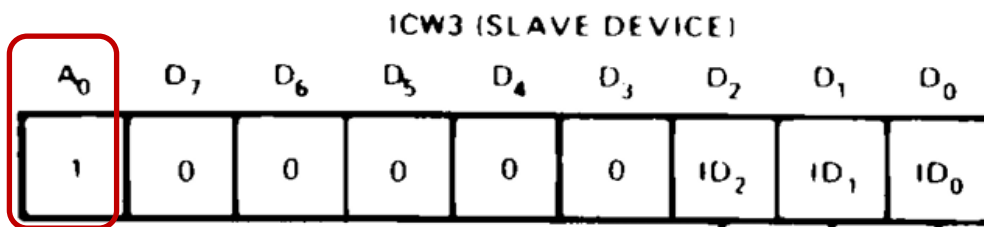
Write 0x04 to 0x21 for  
master PIC. (Line 351)



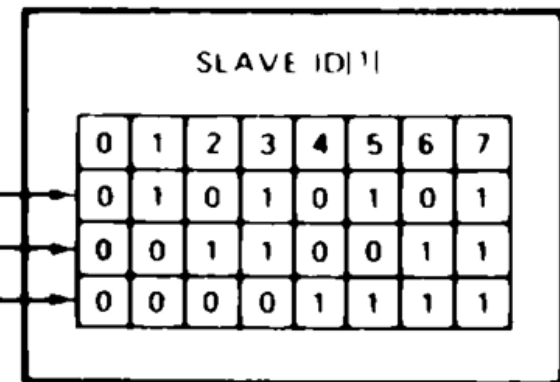
```
outb_p(0x04, 0x21);    /* 8259A-1 (the master) has a slave on IR2 */
```

## ICW3 Slave

0 0 0 0 0 0 1 0



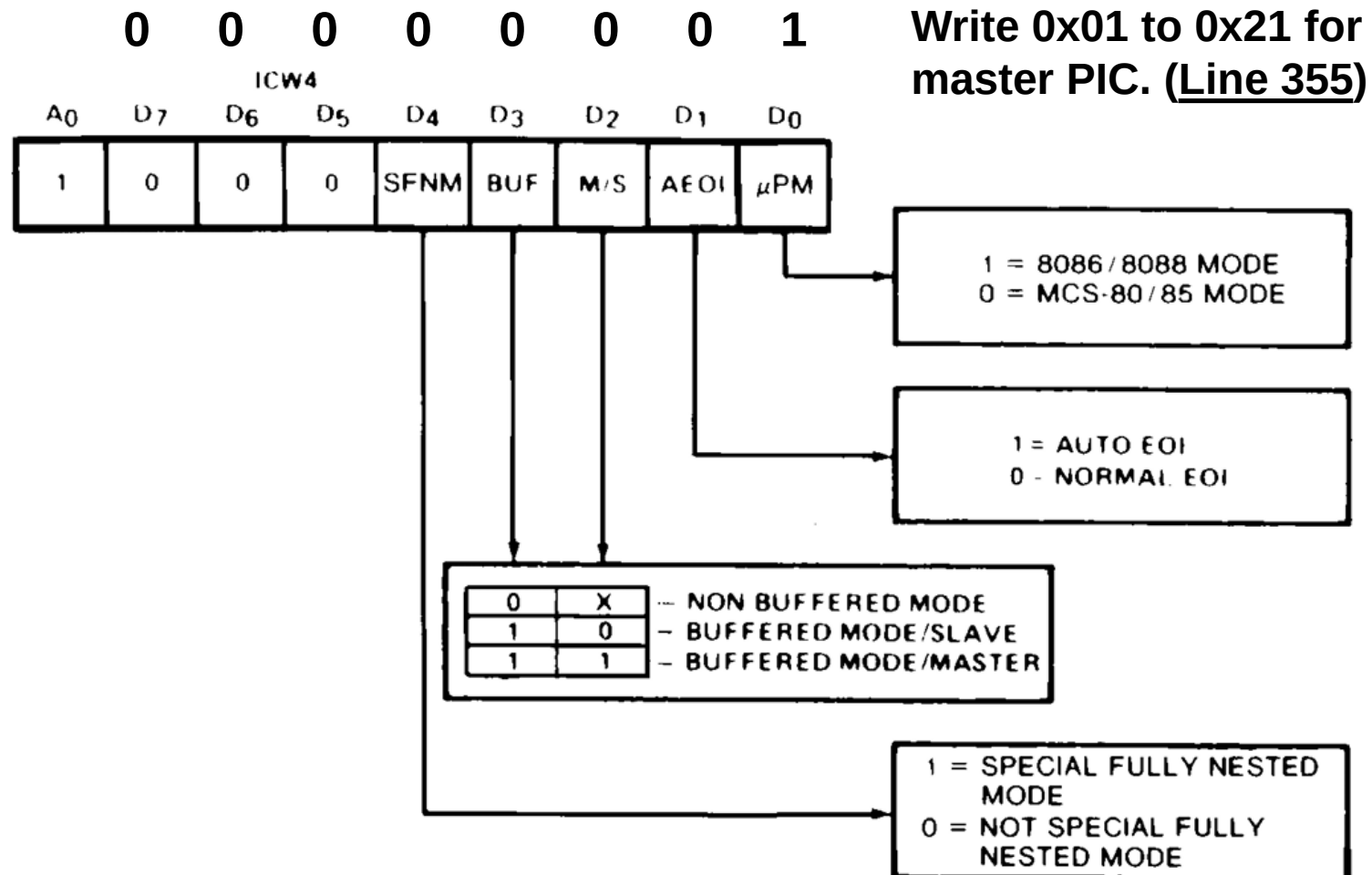
Write 0x02 to 0xA1 for slave PIC. (Line 359)



```
outb_p(0x02, 0xA1);    /* 8259A-2 is a slave on master's IR2 */
```

# ICW4

```
outb_p(0x01, 0x21);    /* master expects normal EOI */
```



# Linux 8259A Initialization

```
void init_8259A(int auto_eoi)
{
    unsigned long flags;

    i8259A_auto_eoi = auto_eoi;
    spin_lock_irqsave(&i8259A_lock, flags);

    outb(0xff, 0x21);      /* mask all of 8259A-1 */
    outb(0xff, 0xA1);      /* mask all of 8259A-2 */

    /*
     * outb_p - this has to work on a wide range of PC hardware.
     */
    outb_p(0x11, 0x20);     /* ICW1: select 8259A-1 init */
    outb_p(0x20 + 0, 0x21); /* ICW2: 8259A-1 IR0-7 mapped to 0x20-0x27 */
    outb_p(0x04, 0x21);     /* 8259A-1 (the primary) has a secondary on IRQ2
    if (auto_eoi)
        outb_p(0x03, 0x21);
    else
        outb_p(0x01, 0x21);

    outb_p(0x11, 0xA0);     /* ICW1: select 8259A-2 init */
    outb_p(0x20 + 8, 0xA1); /* ICW2: 8259A-2 IR0-7 mapped to 0x28-0x2f */
    outb_p(0x02, 0xA1);     /* 8259A-2 is a secondary on primary's IRQ2
    outb_p(0x01, 0xA1);

    if (auto_eoi)
        /*
         * in AEOI mode we just have to mask the interrupt
         * when acking.
         */
        i8259A_irq_type.ack = disable_8259A_irq;
    else
        i8259A_irq_type.ack = mask_and_ack_8259A;

    udelay(100);           /* wait for 8259A to initialize */

    outb(cached_21, 0x21);
    outb(cached_A1, 0xA1);

    spin_unlock_irqrestore(&i8259A_lock, flags);
}
```

# *Comments on Linux' 8259A Initialization Code*

1. What is the auto\_eoi parameter? **always = 0 in Linux**
2. Four initialization control words to set up primary 8259A
3. Four initialization control words to set up secondary 8259A

| port(A=?) |   | info contained in Initialization Control Word                                        |
|-----------|---|--------------------------------------------------------------------------------------|
| ICW1      | 0 | start init, edge-triggered inputs, cascade mode, 4 ICWs                              |
| ICW2      | 1 | high bits of vector #                                                                |
| ICW3      | 1 | primary PIC: bit vector of secondary PIC;<br>secondary PIC: input pin on primary PIC |
| ICW4      | 1 | ISA=x86, normal/auto EOI                                                             |



# *Comments on Linux' 8259A Initialization Code (cont.)*

---

5. What does the “**\_p**” mean on the “outb” macros?
  - add PAUSE instruction after OUTB; “REP NOP” prior to P4
  - delay needed for old devices that cannot handle processor's output rate
6. Critical section spans the whole function
  - avoid other 8259A interactions during initialization sequence
  - (device protocol requires that four words be sent in order)
7. Why **use \_irqsave** for critical section?
  - this code called from other interrupt initialization routines
  - which may or may not have cleared IF on processor